

CRABS: Cryptographic Attribute-Based State Machines

A Protocol for Self-Sovereign Attribute-Based Authorization with Convergent State

Victor J. Morrow

Prometheus

victor.j.morrow@gmail.com

Abstract

Attribute-Based Encryption (ABE) provides fine-grained access control by associating decryption keys with sets of attributes. However, ABE suffers from a fundamental limitation: only a central authority with the master secret key can issue or modify attributes. This creates a single point of trust, hinders self-sovereign identity, and makes dynamic attribute management impractical. We present CRABS, a protocol that unifies ABE with Conflict-free Replicated Data Types (CRDTs) and protocol state machines. In CRABS, *attributes themselves are the state of a state machine*, and changing an attribute is a protocol-governed transition. This eliminates the central authority problem while preserving ABE’s cryptographic guarantees. We describe the protocol’s architecture, algorithms, security properties, and limitations, and discuss applications to decentralized organizations, resource-constrained environments, and self-sovereign identity systems.

1 Introduction

1.1 The Problem

Attribute-Based Encryption (ABE), introduced by Sahai and Waters [1], revolutionized cryptographic access control by allowing encryption policies to reference descriptive attributes rather than specific identities. A ciphertext encrypted under the policy (`role: manager AND department: engineering`) can be decrypted by any user holding a key with those attributes—without the encryptor knowing the recipients in advance.

Despite its power, ABE has a critical operational limitation:

Only the authority holding the Master Secret Key (MSK) can issue, modify, or revoke attributes.

This creates several problems:

1. **Centralized trust**—The authority is a single point of compromise. A malicious or compromised authority can issue arbitrary attributes to any user.
2. **Static attribute sets**—Attributes are typically issued at key generation time and are difficult to change. A user who changes departments, gets married, or is promoted must obtain an entirely new key from the authority.
3. **No audit trail**—Attribute changes are not recorded. There is no cryptographic proof of when or why an attribute was granted or revoked.

4. **No self-sovereignty**—Users cannot assert their own attributes (e.g., a phone number, a nickname) without going through the central authority.
5. **Revocation is hard**—Revoking a user’s attributes typically requires re-encrypting all data or maintaining complex revocation lists.

1.2 Our Approach

We observe that these problems arise from treating attributes as *static cryptographic material* rather than as *dynamic state*. We propose a fundamental shift:

Attributes are not keys. Attributes are state.

In CRABS, the set of all users and their attributes is the state of a **protocol state machine**—the Attribute Machine. Changing an attribute (granting a role, verifying an identity, self-asserting a phone number) is a **state transition** governed by the same protocol rules as any other operation. The ABE keys issued to users are *derived from the current state* and become invalid when the state changes.

This approach:

- Eliminates the central authority—the *protocol* is the authority
- Provides a full audit trail of every attribute change
- Allows users to self-assert attributes within policy constraints
- Makes revocation a natural consequence of state transitions
- Preserves ABE’s cryptographic guarantees (collusion resistance, fine-grained policies)

1.3 Contributions

This paper makes the following contributions:

1. **The CRABS protocol**—A unified architecture combining ABE, CRDTs, and protocol state machines for self-sovereign attribute management.
2. **The Attribute Machine**—A state machine whose state *is* the attribute universe, governed by the same protocol as application-level machines.
3. **Atomic multi-resource locking**—A composable locking primitive for operations on scarce resources, with timeout-based liveness guarantees.
4. **Key versioning**—A mechanism for deriving ABE keys from state versions, enabling seamless attribute updates without re-encrypting data.
5. **Dual privacy modes**—Support for both auditable (Mode A) and privacy-preserving (Mode B) operation signatures.

2 Related Work

2.1 Attribute-Based Encryption

ABE was introduced by Sahai and Waters [1] as “Fuzzy Identity-Based Encryption,” which allowed error-tolerant identity matching. Goyal et al. [2] formalized Key-Policy ABE (KP-ABE), where ciphertexts are tagged with attributes and user keys embed policies. Bethencourt,

Sahai, and Waters [3] introduced Ciphertext-Policy ABE (CP-ABE), the more intuitive variant where ciphertexts embed policies and user keys carry attributes.

Limitation: All these schemes require a trusted authority to generate and manage keys. Multi-authority ABE [4] and decentralized ABE [5] distribute trust across multiple authorities, but still require each authority to manage its own attribute domain. No existing ABE scheme treats attributes as dynamic, protocol-governed state.

2.2 Attribute-Based Signatures

Attribute-Based Signatures (ABS), introduced by Maji et al. [6], allow a user to sign a message with a policy over attributes, proving that *some* user with satisfying attributes endorsed the message, without revealing which user. Okamoto and Takashima [7] provided efficient constructions for circuits.

Limitation: ABS implementations are rare and primarily exist in research libraries. Production-ready ABS libraries are not widely available. CRABS approximates ABS using ABE-gated ECDSA signatures, achieving the same functional properties with battle-tested primitives.

2.3 Conflict-Free Replicated Data Types (CRDTs)

CRDTs, formalized by Shapiro et al. [8], are data structures that can be replicated across multiple nodes and merged without conflicts. Types include G-Counters (grow-only counters), PN-Counters (positive-negative counters), OR-Sets (observed-remove sets), LWW-Registers (last-writer-wins registers), and RGA (Replicated Growable Arrays) for collaborative text editing [9].

Relevance: CRDTs provide the convergence layer for CRABS. Operations can be applied in any order across nodes, and the state will converge to the same result. This eliminates the need for global consensus.

2.4 Protocol State Machines and Session Types

Protocol state machines, also known as typestate [10] or session types [11], specify valid sequences of operations on a resource. The resource’s current state determines which operations are permitted next. This is the foundation of CRABS’s resource locking protocol.

Relevance: CRABS extends typestate to a decentralized, CRDT-based setting where multiple participants can observe and verify protocol compliance.

2.5 Capability-Based Security

Capability systems [12] grant access based on possession of a capability token rather than identity. CRABS’s capability vault—where ABE-encrypted ECDSA signing keys serve as capabilities—is a direct application of this principle to the attribute-based setting.

2.6 Self-Sovereign Identity

Self-Sovereign Identity (SSI) [13] is a paradigm where individuals control their own digital identities without relying on central authorities. Verifiable Credentials (VCs) and Decentralized Identifiers (DIDs) are key building blocks.

Relevance: CRABS’s Attribute Machine provides a cryptographic foundation for SSI, where attributes are self-asserted or verified by peers, and all changes are recorded in an auditable transition log.

3 The CRABS Protocol

3.1 Architecture Overview

CRABS consists of three layers, as shown in Figure 1.

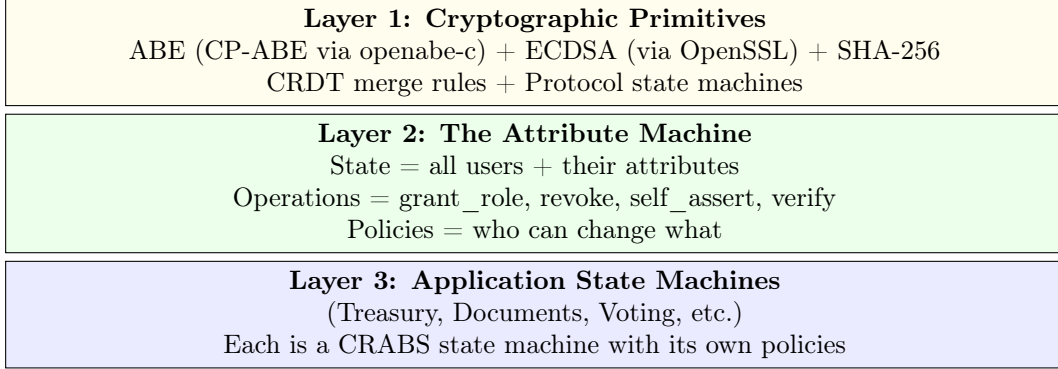


Figure 1: The three-layer CRABS architecture.

The Attribute Machine is the foundation. Application machines reference its state for authorization decisions. All machines follow the same protocol.

3.2 Data Model

The state of a CRABS machine is a **bag of typed data items**:

```
State = {
  items: {
    "treasury": {
      type: RESOURCE,
      crdt_type: PN_COUNTER,
      value: {ETH: 100},
      protocol_state: "idle",
      invariants: [{type: GREATER_THAN, value: 0}]
    },
    "members": {
      type: COLLECTION,
      crdt_type: OR_SET,
      value: {alice, bob, carol},
      protocol_state: "idle",
      invariants: [{type: UNIQUE}]
    },
    "seats": {
      type: RESOURCE,
      crdt_type: G_COUNTER,
      value: 50,
      protocol_state: "idle",
      invariants: [
        {type: GREATER_THAN, value: 0},
        {type: DIVISIBLE_BY, value: 2}
      ]
    }
  },
  policies: {
    "transfer": "role:admin",
    "vote": "role:member",
  }
}
```

```

    "lock": "role:admin OR role:member",
    "unlock": "role:admin OR role:member"
  },
  lock_manager: { ... },
  transition_log: [...],
  version: 42
}

```

3.3 Data Types

CRABS defines a small set of built-in data types, each with a CRDT merge strategy:

Type	CRDT Strategy	Lock?	Example
COUNTER	G-Counter (grow-only)	No	Vote counts, reputation
PN_COUNTER	PN-Counter (pos/neg)	For transfers	Balances, inventory
SET	OR-Set (observed-remove)	No	Member lists, permissions
2P_SET	Two-phase set	No	Blacklists, revoked keys
REGISTER	LWW-Register (last-writer-wins)	No	Profile info, titles
DOCUMENT	RGA (replicated growable array)	No	Collaborative text
RESOURCE	PN-Counter + lock protocol	Yes	Treasury, seats, contracts
CUSTOM	User-defined callbacks	Configurable	Application-specific

3.4 Protocol States for Resources

Resources (items with inherent scarcity) follow a strict protocol state machine, depicted in Figure 2.

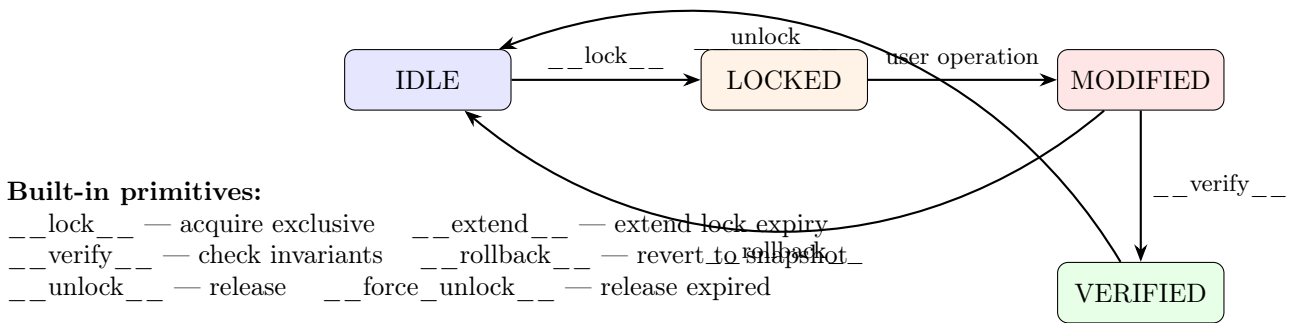


Figure 2: The resource protocol state machine. Resources transition through IDLE → LOCKED → MODIFIED → VERIFIED → IDLE.

3.5 The Operation

Operations are the atomic unit of state change. They are completely generic—the state machine does not interpret operation semantics, only protocol correctness.

```

Operation = {
  type: "transfer",           // Arbitrary string
  payload: {amount: 10, to: "bob"}, // Application-defined

  resources: ["treasury", "seats"], // Data items touched
  required_state: ["locked", "idle"], // Protocol state per resource
  next_state: ["modified", "modified"], // Target state per resource
}

```

```

lock_claims: [
    // Proof of lock ownership
    {resource: "treasury", token: 0xA3F7...},
    {resource: "seats", token: 0xB7E2...}
],

policy: "role:admin", // ABE policy for
    authorization
signature: 0x..., // ECDSA signature
signer_id: "carol", // Mode A, or "" for Mode B

lamport_time: 2042, // Causal ordering
node_id: "node_7" // Originating node
}

```

3.6 The Attribute Machine

The Attribute Machine is a CRABS state machine whose state is the set of all users and their attributes:

```

AttributeMachine State = {
  users: {
    "alice": {
      attributes: {
        role: "writer",
        dept: "engineering",
        phone: "555-0123", // Self-asserted
        name: "Alice Smith" // Verified by HR
      },
      public_key: 0xABC...,
      status: "active",
      key_version: 42
    },
    "bob": { ... }
  },
  policies: {
    "grant_role": "role:admin",
    "revoke_role": "role:admin",
    "verify_identity": "role:admin OR role:verifier",
    "self_assert": "role:member",
    "change_policy": "role:admin AND weight >= 3"
  }
}

```

Key insight: The Attribute Machine follows the *same protocol* as any other CRABS machine. There is no privileged “authority”—only protocol rules.

3.7 Key Derivation and Versioning

ABE keys are derived from the current state of the Attribute Machine:

```

KeyEnvelope = {
  user_id: "alice",
  state_version: 42, // Which state version this
    is for
  attributes_hash: SHA256(alice_attrs), // Hash of attributes at that
    version
}

```

```

    sk_abe: ABE_keygen(MSK, "role:writer|dept:engineering|...")
}

```

Verification during any operation:

```

function verify_key_is_current(state, key):
    if key.state_version != state.version:
        return REJECT("Key is stale. Current version: {state.version}")

    current_hash = SHA256(state.users[key.user_id].attributes)
    if key.attributes_hash != current_hash:
        return REJECT("Attributes have changed since key was issued")

    return ACCEPT

```

When attributes change, users request a key refresh:

```

Operation: __refresh_abe_key__
Resources: [attribute_machine]
Policy: "role:member"
Effect: State machine derives new key from current attributes
       Old key is invalidated (version mismatch)

```

4 Algorithms

4.1 Atomic Lock Acquisition

```

function atomic_acquire_locks(state, resources, lock_tokens, requester)
:
    acquired = []

    // Phase 1: Attempt all
    for each (resource, token) in zip(resources, lock_tokens):
        res = state.items[resource]

        if res.protocol_state != "idle":
            // Resource is locked or unavailable
            goto rollback

        if not check_policy(state, "__lock__", requester):
            goto rollback

        // Acquire
        res.protocol_state = "locked"
        res.lock_token = token
        res.lock_owner = requester
        res.lock_expiry = now() + state.config.max_lock_duration
        res.pre_lock_snapshot = snapshot(res)
        res.lock_extensions = 0
        acquired.append(resource)

    return SUCCESS

rollback:
    for each resource in acquired:
        res = state.items[resource]

```

```
    res.protocol_state = "idle"
    res.lock_token = null
    res.lock_owner = null
    res.pre_lock_snapshot = null

    return LOCK_CONTENTION
```

Virtues:

- Atomic—no partial lock state is ever visible
- Deadlock-free—all-or-nothing semantics
- Policy-gated—only authorized users can lock

Limitations:

- Requires all resources to be known in advance
- Contention increases with number of resources per operation
- No priority ordering—first-come-first-served

4.2 Force Unlock (Timeout)

```
function force_unlock(state, resource, requester):
    res = state.items[resource]

    if now() < res.lock_expiry:
        return LOCK_NOT_EXPIRED

    if not state.config.allow_force_unlock:
        return FORCE_UNLOCK_DISABLED

    // Roll back to pre-lock state
    if res.pre_lock_snapshot != null:
        restore(res, res.pre_lock_snapshot)
        res.pre_lock_snapshot = null

    // Release
    res.protocol_state = "idle"
    res.lock_token = null
    res.lock_owner = null

    // Log
    log(state, {
        type: "__force_unlock__",
        resource: resource,
        forced_by: requester,
        previous_owner: res.lock_owner
    })

    return SUCCESS
```

Virtues:

- Guarantees liveness—no resource can be locked indefinitely
- Anyone can trigger—no single point of failure

- Rollback ensures consistency

Limitations:

- Rollback discards any unverified changes (by design)
- Requires synchronized clocks (or generous timeout margins)
- Malicious nodes could force-unlock immediately after expiry

4.3 Operation Execution

```
function execute_operation(state, op):
    // Step 1: Prune expired locks
    prune_expired_locks(state)

    // Step 2: Verify protocol states
    for each (resource, required) in zip(op.resources, op.
        required_state):
        res = state.items[resource]
        if res.protocol_state != required:
            return PROTOCOL_VIOLATION

    // Step 3: Verify lock claims (for locked resources)
    for each claim in op.lock_claims:
        res = state.items[claim.resource]
        if res.protocol_state == "locked":
            if res.lock_token != claim.token:
                return LOCK_TOKEN_MISMATCH
            if res.lock_owner != op.signer_id and op.signer_id != "":
                return LOCK_OWNER_MISMATCH

    // Step 4: Verify ABE policy
    if not verify_abe_signature(state, op):
        return UNAUTHORIZED

    // Step 5: Verify key is current
    if not verify_key_is_current(state, op.signer_key):
        return KEY_STALE

    // Step 6: Execute handler (application-defined)
    result = op.handler(state, op)
    if result != SUCCESS:
        return result

    // Step 7: Transition protocol states
    for each (resource, next) in zip(op.resources, op.next_state):
        state.items[resource].protocol_state = next

    // Step 8: Log
    log(state, op)
    state.version++

    return SUCCESS
```

Virtues:

- All security checks happen before execution

- Protocol state transitions are atomic with execution
- Full audit trail

Limitations:

- Sequential within a single node (though CRDT merge allows parallel across nodes)
- Handler is application-defined—cannot verify its correctness cryptographically

4.4 CRDT Merge

```
function crdt_merge(state_a, state_b):
    merged = copy(state_a)

    for each (name, item_b) in state_b.items:
        if name not in merged.items:
            merged.items[name] = item_b
        else:
            item_a = merged.items[name]

            switch item_a.crdt_type:
                case G_COUNTER:
                    merged.items[name].value = max(item_a.value, item_b.value)

                case PN_COUNTER:
                    merged.items[name].value.pos =
                        max(item_a.value.pos, item_b.value.pos)
                    merged.items[name].value.neg =
                        max(item_a.value.neg, item_b.value.neg)

                case OR_SET:
                    merged.items[name].value =
                        union(item_a.value, item_b.value)
                    merged.items[name].tombstones =
                        union(item_a.tombstones, item_b.tombstones)

                case LWW_REGISTER:
                    if item_b.timestamp > item_a.timestamp:
                        merged.items[name].value = item_b.value
                        merged.items[name].timestamp = item_b.timestamp

                case RGA:
                    merged.items[name].value =
                        rga_merge(item_a.value, item_b.value)

    // Merge transition logs
    merged.log = merge_sorted_logs(state_a.log, state_b.log)
    merged.version = max(state_a.version, state_b.version)

    return merged
```

Virtues:

- Eventual consistency without consensus
- Deterministic—all nodes converge to the same state

- Handles concurrent operations gracefully

Limitations:

- CRDTs have limited expressiveness (no arbitrary computation)
- Tombstones in OR-Sets grow unboundedly (can be garbage-collected with coordination)
- Last-writer-wins registers lose concurrent updates

4.5 Key Refresh

```
function refresh_abe_key(state, msk, mpk, user_id, requester):
    user = state.users[user_id]

    // Verify requester is the user or an admin
    if requester != user_id
        and not check_policy(state, "refresh_key", requester):
        return UNAUTHORIZED

    // Build attribute string from current state
    attrs = []
    for each (name, value) in user.attributes:
        attrs.append("{name}:{value}")
    attr_string = join(attrs, "|")

    // Generate new key
    sk_abe = ABE_keygen(msk, mpk, attr_string)

    // Create key envelope with current state version
    envelope = {
        user_id: user_id,
        state_version: state.version,
        attributes_hash: SHA256(user.attributes),
        sk_abe: sk_abe
    }

    // Update user's key version
    user.key_version = state.version

    return envelope
```

Virtues:

- Keys are always derived from current state
- Old keys are automatically invalidated (version mismatch)
- No need to re-encrypt existing data—only new operations check key freshness

Limitations:

- Requires users to proactively refresh keys after attribute changes
- Brief window where old key could be used if state version check is delayed
- Key refresh is an online operation (requires communication with a node)

5 Security Analysis

5.1 Threat Model

We assume the following adversary capabilities:

- **Network adversary:** Can observe, delay, reorder, and drop messages
- **Byzantine nodes:** Up to f nodes may behave arbitrarily (but not collude with the ABE authority, which is the protocol itself)
- **Compromised users:** Can attempt to use stale keys, forge signatures, or collude

We do **not** defend against:

- Compromise of the MSK (though the MSK can be rotated via protocol transition)
- Quantum attacks on bilinear pairings (post-quantum variants are future work)
- Sybil attacks on node identity (requires external PKI or proof-of-work)

5.2 Security Properties

Property	Mechanism	Guarantee
Authorization	ABE policy verification	Only users with matching attributes can perform operations
Authentication	ECDSA signatures	Operations are cryptographically bound to a specific key
Non-repudiation	Signed log entries	Past operations cannot be denied
Collusion resistance	ABE key randomization	Two users cannot combine keys to gain unauthorized access
Replay protection	Lamport clocks + nonces	Old messages cannot be replayed
Liveness	Lock timeout + force-unlock	No resource can be locked indefinitely
Confidentiality	ABE encryption	Only authorized users can decrypt operation payloads
Privacy (Mode B)	Optional signer omission	Verifier learns only that <i>some</i> authorized user signed
Auditability (Mode A)	Signer identity included	Every operation is attributable to a specific user

5.3 Key Compromise

If a user's ECDSA signing key is compromised:

1. The user reports the compromise via a signed operation
2. An admin revokes the user's key via a state transition
3. The user is issued a new key pair
4. The old key is added to a revocation set (2P-Set)
5. Any operation signed with the old key is rejected

If the MSK is compromised:

1. The MSK is rotated via a protocol transition (requires threshold of admins)
2. All users refresh their ABE keys
3. Old ciphertexts remain secure (the old MSK is discarded, not exposed)
4. New ciphertexts use the new MPK

6 Use Cases

6.1 Decentralized Autonomous Organizations (DAOs)

CRABS provides a natural foundation for DAOs where membership is defined by attributes rather than token holdings:

```
DAO State = {
  treasury: RESOURCE(PN_COUNTER, {ETH: 1000}),
  members: SET(OR_SET, {alice, bob, carol}),
  proposals: CUSTOM(...),
  policies: {
    "create_proposal": "role:member",
    "vote": "role:member",
    "execute": "role:admin OR (votes >= 3 AND role:member)",
    "add_member": "role:admin",
    "remove_member": "role:admin AND weight >= 2",
    "spend_funds": "role:admin AND weight >= 3"
  }
}
```

Advantages over token-based DAOs:

- Plutocracy is avoided—one member, one vote
- Roles enable fine-grained permissions
- No gas costs for voting
- Privacy-preserving voting (Mode B)

6.2 Collaborative Document Review

A document review workflow with dynamic policy updates:

```
Document State = {
  content: DOCUMENT(RGA, "..."),
  status: REGISTER(LWW, "draft"),
  policies: {
    "edit": "role:author", // Only author can edit
    "submit": "role:author", // Author submits for review
    "review": "role:reviewer", // Reviewer reviews
    "approve": "role:editor", // Editor approves
    "publish": "role:admin" // Admin publishes
  }
}

// Transition: submit for review
// Effect: status -> "in_review"
//           policies.edit -> "role:reviewer"
//           policies.request_changes -> "role:reviewer"
```

6.3 Self-Sovereign Identity

The Attribute Machine enables a fully self-sovereign identity system:

```
AttributeMachine State = {
  users: {
    "alice": {
      attributes: {
        // Self-asserted (any member can add)
        "nickname": "Ali",
        "phone": "555-0123",

        // Verified (requires role:verifier)
        "name": "Alice Smith (verified by HR)",
        "passport": "hash:0x... (verified by KYC)"
      }
    }
  },
  policies: {
    "self_assert":      "role:member",
    "verify":           "role:verifier",
    "revoke_claim":     "role:admin OR self"
  }
}
```

Users control their own attributes while relying on verifiers for high-assurance claims. All changes are logged and auditable.

6.4 Supply Chain Management

Each shipment is a resource with a protocol:

```
Shipment Protocol:
  IDLE -> __lock__ -> LOCKED -> inspect -> MODIFIED
           -> __verify__ -> VERIFIED -> __unlock__ -> IDLE
  IDLE -> __lock__ -> LOCKED -> transfer -> MODIFIED
           -> __verify__ -> VERIFIED -> __unlock__ -> IDLE
  MODIFIED -> __rollback__ -> IDLE (if inspection fails)

Policies:
  "inspect":  "role:inspector AND org:customs"
  "transfer": "role:logistics AND org:carrier"
  "lock":     "role:inspector OR role:logistics"
```

Different organizations (customs, carriers, warehouses) have different roles, and the protocol ensures that custody transfers are atomic and auditable.

7 Limitations

7.1 Cryptographic Limitations

Limitation	Impact	Mitigation
MSK is a single point of compromise	If MSK leaks, attacker can forge any attribute	Threshold MSK schemes (future work)
Pairing operations are expensive	ABE operations are 10–100× slower than AES	Use ABE only for policy enforcement, not bulk encryption
Key size grows with attributes	Each attribute adds a group element	Limit attributes to ~ 50 per user in practice
No post-quantum security	Bilinear pairings vulnerable to Shor’s algorithm	Lattice-based ABE variants exist but are immature

7.2 Protocol Limitations

Limitation	Impact	Mitigation
No global consensus	State may diverge temporarily	CRDT merge guarantees eventual convergence
Lock contention	Multiple users competing for same resource	Short lock durations, exponential back-off
Tombstone growth	OR-Set tombstones grow unboundedly	Periodic garbage collection with coordination
Clock synchronization	Lock expiry depends on clocks	Use generous timeouts ($2\times$ network latency)
Online key refresh	Users must contact a node after attribute change	Can be batched or made asynchronous

7.3 Expressiveness Limitations

Limitation	Impact	Mitigation
No Turing-complete computation	Cannot express arbitrary smart contracts	CRABS is declarative by design—policies are simpler and auditable
CRDTs limit data types	Not all data structures have CRDT equivalents	Custom types with user-defined merge functions
No automatic execution	Transitions require user action	Can be combined with cron-like schedulers

8 Future Work

8.1 Threshold MSK

The Master Secret Key remains a single point of compromise. A threshold scheme where the MSK is split across n nodes, requiring k to sign, would eliminate this vulnerability. This would make the Attribute Machine truly decentralized.

8.2 Post-Quantum CRABS

Lattice-based ABE schemes are an active research area. Porting CRABS to lattice assumptions would provide post-quantum security. The protocol architecture is agnostic to the underlying cryptographic primitives.

8.3 Hierarchical Attribute Machines

An organization might have multiple Attribute Machines (one per department) with cross-machine trust relationships. A machine in engineering might trust a machine in HR for `name` attributes but not for `role` attributes.

8.4 Garbage Collection for CRDT Tombstones

OR-Set tombstones grow unboundedly. A garbage collection protocol that safely removes tombstones after all nodes have observed the removal would be valuable for long-running state machines.

8.5 Lightweight Clients

The current design assumes clients can perform ABE operations. Lightweight clients (mobile, IoT) might delegate ABE operations to a trusted proxy while retaining ECDSA signing locally.

8.6 Formal Verification

The CRABS protocol’s safety properties (no invalid state transitions, invariant preservation, CRDT convergence) could be formally verified using a proof assistant such as Coq or Isabelle/HOL.

8.7 Interoperability with Existing ABE Systems

A bridge between CRABS and existing ABE deployments would allow gradual migration. Ciphertexts encrypted under a traditional ABE authority could be re-encrypted under CRABS policies.

8.8 Economic Incentives for Force-Unlock

Force-unlock is a public good—anyone can do it, but no one is incentivized to. A small reward (taken from the lock holder’s stake) for force-unlocking expired locks would ensure liveness in practice.

9 Conclusion

CRABS addresses the fundamental limitation of Attribute-Based Encryption—the central authority problem—by treating attributes as dynamic, protocol-governed state rather than static cryptographic material. The Attribute Machine, a CRABS state machine whose state *is* the attribute universe, eliminates the need for a trusted authority while preserving ABE’s cryptographic guarantees.

By combining ABE with CRDTs, protocol state machines, and atomic multi-resource locking, CRABS provides a unified framework for decentralized authorization that is:

- **Self-sovereign**—Users control their own attributes within policy constraints
- **Auditable**—Every attribute change is a logged, signed transition

- **Convergent**—CRDT merge ensures eventual consistency without consensus
- **Liveness-guaranteed**—Lock timeouts prevent indefinite resource holding
- **Privacy-preserving**—Optional signer anonymity (Mode B)

The protocol is implementable today with existing libraries (openabe-c for ABE, OpenSSL for ECDSA, custom CRDT merge logic) and is suitable for applications ranging from DAOs and document review to supply chain management and self-sovereign identity.

References

- [1] A. Sahai and B. Waters, “Fuzzy Identity-Based Encryption,” *EUROCRYPT*, 2005.
- [2] V. Goyal, O. Pandey, A. Sahai, and B. Waters, “Attribute-Based Encryption for Fine-Grained Access Control of Encrypted Data,” *CCS*, 2006.
- [3] J. Bethencourt, A. Sahai, and B. Waters, “Ciphertext-Policy Attribute-Based Encryption,” *IEEE S&P*, 2007.
- [4] M. Chase, “Multi-Authority Attribute Based Encryption,” *TCC*, 2007.
- [5] A. Lewko and B. Waters, “Decentralizing Attribute-Based Encryption,” *EUROCRYPT*, 2011.
- [6] H. Maji, M. Prabhakaran, and M. Rosulek, “Attribute-Based Signatures,” *CT-RSA*, 2011.
- [7] T. Okamoto and K. Takashima, “Efficient Attribute-Based Signatures for Non-Monotone Predicates in the Standard Model,” *PKC*, 2011.
- [8] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, “Conflict-Free Replicated Data Types,” *SSS*, 2011.
- [9] R. Levien, “Replicated Growable Array (RGA),” 2016. <https://github.com/xi-editor/xi-editor>
- [10] R. DeLine and M. Fähndrich, “Typestates for Objects,” *ECOOP*, 2004.
- [11] K. Honda, V. Vasconcelos, and M. Kubo, “Language Primitives and Type Discipline for Structured Communication-Based Programming,” *ESOP*, 1998.
- [12] J. Dennis and E. Van Horn, “Programming Semantics for Multiprogrammed Computations,” *CACM*, 1966.
- [13] C. Allen, “The Path to Self-Sovereign Identity,” 2016. <http://www.lifewithalacrity.com/2016/04/the-path-to-self-sovereign-identity.html>

CRABS: Hard shell. Sharp pincers. No backdoors.